

課題 4.1

ガウスの消去法とヤコビ法のプログラムを書き，以下の連立 1 次方程式をそれぞれの方法で計算しなさい。計算できない場合には，なぜ計算できないか検討しなさい。

$$(1) \begin{bmatrix} 4 & 1 & 1 \\ 2 & 6 & 1 \\ 1 & 2 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ -8 \\ 7 \end{bmatrix}$$

$$(2) \begin{bmatrix} 4 & 2 & -1 \\ 1 & -3 & 1 \\ 3 & 4 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 8 \\ 8 \end{bmatrix}$$

$$(3) \begin{bmatrix} 0 & 1 & 2 \\ 1 & 0 & 3 \\ 3 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ -3 \end{bmatrix}$$

1. ライブラリーと定数宣言

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

2. 関数宣言

```
void gaussian(double a[3][3], double b[3]);
void jacobian(double a[3][3], double b[3]);
```

① 関数 gaussian(double a[3][3], double b[3])

引数 実数行列 3×3 a
 実数配列 3 次元 b

$ax = b$ をガウス消去法で解いて画面に出力する。途中で対角成分が 0 となったとき、計算できないと警告メッセージを出して終了する。

```
void gaussian(double a[3][3], double b[3]){
    double A[3][3];
    double x[3], c;
    int i, j, k;

    for(i = 0; i < 3; i++){
        for(j = 0; j < 3; j++){
            A[i][j] = a[i][j];
            x[i] = b[i];
        }
    }
}
```

```
//forward
for(i = 0; i < 3; i++){
    if(A[i][i] == 0){
        printf("A diagonal element is 0; Cannot calculate\n");
        return;
    }
    for(j = i + 1; j < 3; j++){
        c = A[j][i] / A[i][i];
        for(k = i; k < 3; k++){
            A[j][k] = A[j][k] - c * A[i][k];
        }
        x[j] = x[j] - c * x[i];
    }
}

//backward
for(i = 2; i >= 0; i--){
    for(j = i - 1; j >= 0; j--){
        c = A[j][i] / A[i][i];
        A[j][i] = A[j][i] - c * A[i][i];
        x[j] = x[j] - c * x[i];
    }
    x[i] = x[i] / A[i][i];
}

printf("Gaussian:\tx1 = %.0f\tx2 = %.0f\tx3 = %.0f\n", x[0], x[1], x[2]);
}
```

② 関数 jacobian(double a[3][3], double b[3])

引数	実数行列 3×3	a
	実数配列 3 次元	b

$\mathbf{ax} = \mathbf{b}$ をヤコビ消去法で解いて画面に出力する。最初に 0 の対角成分があるか確認する。0 の対角成分があれば計算できないと警告メッセージを出して終了する。

```
void jacobian(double a[3][3], double b[3]){
    double x_n[3], x[3];
    double err = 1e-8;
    int i, j;

    for(i = 0; i < 3; i++)
        // A(i,i) != 0
        if(a[i][i] == 0) {
            printf("A diagonal element is 0; Cannot calculate\n");
        }
}
```

```

        return;
    }

    for(j = 0; j < 3; j++)        // initialize x0 with random values
        x_n[j] = ((double) rand()) / RAND_MAX;

    while(! (fabs(x_n[0] - x[0]) < err && fabs(x_n[1] - x[1]) < err && fabs(x_n[2] -
x[2]) < err)){
        x[0] = x_n[0];
        x[1] = x_n[1];
        x[2] = x_n[2];

        x_n[0] = (b[0] - (a[0][1] * x[1] + a[0][2] * x[2])) / a[0][0];
        x_n[1] = (b[1] - (a[1][0] * x[0] + a[1][2] * x[2])) / a[1][1];
        x_n[2] = (b[2] - (a[2][0] * x[0] + a[2][1] * x[1])) / a[2][2];
    }

    printf("Jacobian:\tx1 = %.0f\tx2 = %.0f\tx3 = %.0f\n", x_n[0], x_n[1], x_n[2]);
}

```

3. メイン関数

```

int main(int argc, char** argv){
    double a1[3][3] = {{4,1,1},{2,6,1},{1,2,5}}, b1[3] = {4, -8, 7};
    double a2[3][3] = {{4,2,-1},{1,-3,1},{3,4,2}}, b2[3] = {3, 8, 8};
    double a3[3][3] = {{0,1,2},{1,0,3},{3,1,0}}, b3[3] = {2, 2, -3};

    printf("1)\n");
    gaussian(a1,b1);
    jacobian(a1,b1);
    printf("2)\n");
    gaussian(a2,b2);
    jacobian(a2,b2);
    printf("3)\n");
    gaussian(a3,b3);
    jacobian(a3,b3);

    return 0
}

```

4. 結果

1)

Gaussian: x1 = 1 x2 = -2 x3 = 2

Jacobian: x1 = 1 x2 = -2 x3 = 2

2)

Gaussian: $x_1 = 2 \quad x_2 = -1 \quad x_3 = 3$

Jacobian: $x_1 = 2 \quad x_2 = -1 \quad x_3 = 2$

3)

A diagonal element is 0; Cannot calculate

A diagonal element is 0; Cannot calculate

課題 4.2

オイラー法とルンゲ・クッタ法のプログラムを書き、1 階の常微分方程式 $y' = x + y$ をそれぞれの方法で計算しなさい。また、解析解と比較したときの誤差特性をグラフ化し検討しなさい。

ただし、初期条件を $y(0) = 1$ 、区間を $[0,1]$ 、分割数を 10 とする。

1. ライブラリーと定数宣言

```
#include <stdio.h>
#include <math.h>
#define N 10
```

N は分割数を表す

2. 関数宣言

```
double f(double x, double y);
double Y(double x);           // solution for dy/dx = f(x,y)
void euler(double x, double y, double a, double b);
void runge(double x, double y, double a, double b);
```

関数 f は $\frac{dy}{dx} = f(x,y)$ を満たすものとする。関数 Y はその解である。関数 euler と runge はそれぞれオイラー法とルンゲ・クッタ法による関数である。

③ 関数 double f(double x, double y)

引数 実数 x, y

戻り値 実数 $x + y$

```
double f(double x, double y){ return x + y; }
```

④ 関数 double Y(double x)

$\frac{dy}{dx} = x + y$ は一階線形微分方程式であり、 $\frac{dy}{dx} - y = x$ に変形して両方に e^{-x} を掛けると、 $e^{-x} \frac{dy}{dx} - e^{-x} y = x e^{-x}$ となる。

$$\text{左辺} \quad \int e^{-x} \frac{dy}{dx} - e^{-x} y \, dx = \int e^{-x} \frac{dy}{dx} + \frac{d e^{-x}}{dx} y \, dx = \int \frac{d}{dx} y e^{-x} \, dx = y e^{-x} + C_1$$

$$\text{右辺} \quad \int x e^{-x} \, dx = -x e^{-x} + \int e^{-x} \, dx = -x e^{-x} - e^{-x} + C_2$$

$$y e^{-x} + C_1 = -x e^{-x} - e^{-x} + C_2$$

$$y = -x - 1 + C e^x$$

$$y(0) = 1 \text{ より } 1 = -0 - 1 + C e^0 \Rightarrow C = 2 \text{ で } y(x) = -x - 1 + 2e^x$$

引数 実数 x

戻り値 実数 $-x - 1 + Ce^x$ の値

```
double Y(double x){ return 2 * exp(x) - x - 1; }
```

⑤ 関数 void euler(double x, double y, double a, double b)

引数 実数 x 変数 $x = x_0$ の初期値

 実数 y 変数 $y(x_0)$ の初期値

 実数 a, b 積分範囲

刻み幅を $h = \frac{b-a}{N}$ から計算し、ルンゲ・クッタ法による微分方程式を近似し、ファイル ./euler.dat に出力する。

```
void euler(double x, double y, double a, double b){
    int i;
    double h = (b - a) / N;
    FILE* output = fopen("./euler.dat", "w+");
    for(i = 0; i < N; i++){
        y = y + h * f(x,y);
        x = x + h;
        fprintf(output, "%.10f\t%.10f\t%.10f\t%.10f\n", x, y, Y(x), fabs(Y(x) - y));
    }
    fclose(output);
}
```

⑥ 関数 double runge(double x, double y, double a, double b)

引数 実数 x 変数 $x = x_0$ の初期値

 実数 y 変数 $y(x_0)$ の初期値

 実数 a, b 積分範囲

刻み幅を $h = \frac{b-a}{N}$ から計算し、ルンゲ・クッタ法による微分方程式を近似し、ファイル ./runge.dat に出力する。

```
void runge(double x, double y, double a, double b){
    int i;
    double h = (b - a) / N;
    double k1, k2, k3, k4;
    FILE* output = fopen("./runge.dat", "w+");
    for(i = 0; i < N; i++){
        k1 = h * f(x,y);
        k2 = h * f(x + h / 2, y + k1 / 2);
        k3 = h * f(x + h / 2, y + k2 / 2);
        k4 = h * f(x + h, y + k3);
        y = y + (k1 + 2 * k2 + 2 * k3 + k4) / 6;
        x = x + h;
    }
}
```

```

    fprintf(output, "%.10f\t%.10f\t%.10f\t%.10f\n", x, y, Y(x), fabs(Y(x) - y));
}
fclose(output);
}

```

3. メイン関数

```

int main(int argc, char** argv){
    double x0 = 0, y0 = 1;
    double a = 0, b = 1;
    euler(x0, y0, a, b);
    runge(x0, y0, a, b);
}

```

4. 実行結果

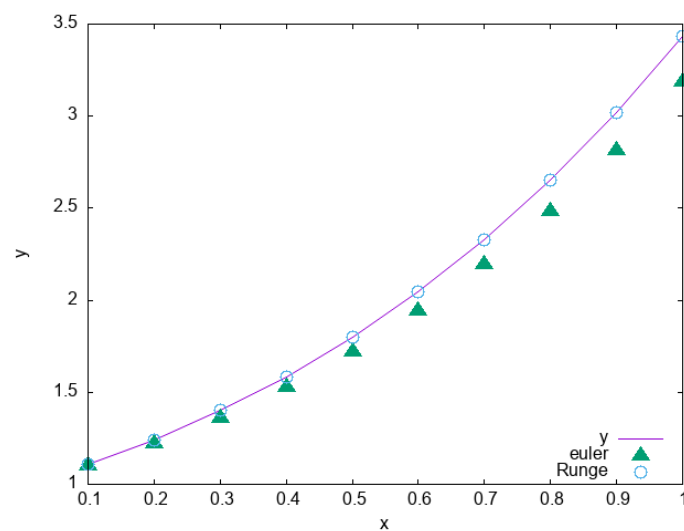


図 2.1 オイラー法とルンゲ・クッタ法による計算結果

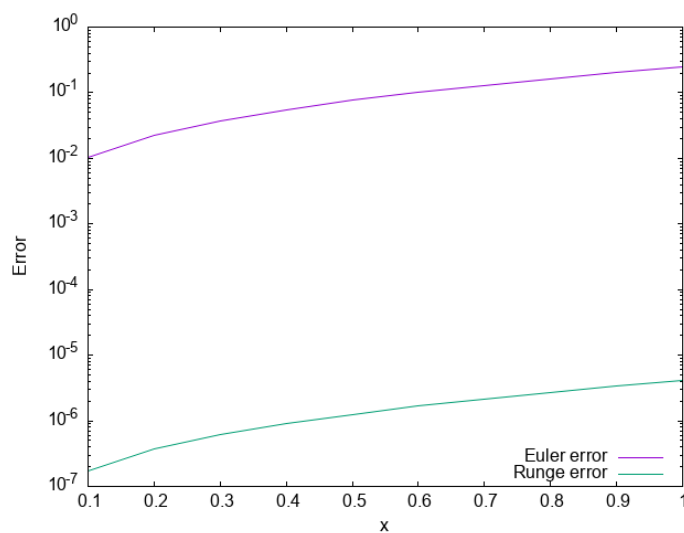


図 2.2 オイラー法とルンゲ・クッタ法による計算の誤差

図 2.1 から見れば、ルンゲ・クッタ法による計算が y の曲線と一致している一方、オイラー法による計算が徐々に離れていく。図 2.2 から見れば、オイラー法による誤差がルンゲ・クッタ法による誤差より圧倒的に大きいことが分かる。両方とも、初期値から離れれば離れるほど誤差が増大している。

課題 4.3

課題 4.2 について、オイラー法とルンゲ・クッタ法の刻み幅 h による誤差特性をグラフ化し検討しなさい。数値解の誤差は、 $x = 1.0$ における値を評価すればよい

1. ライブラリーと定数宣言

```
#include <stdio.h>
#include <math.h>
```

2. 関数宣言

```
double f(double x, double y);
double Y(double x);           // solution for dy/dx = f(x,y)
void euler(int N, double x, double y, double X);
void runge(int N, double x, double y, double X);
```

関数 f は $\frac{dy}{dx} = f(x,y)$ を満たすものとする。関数 Y はその解である。関数 `euler` と `runge` はそれぞれオイラー法とルンゲ・クッタ法による関数である。

① 関数 `double f(double x, double y)`

引数 実数 x, y
戻り値 実数 $x + y$

```
double f(double x, double y){ return x + y; }
```

② 関数 `double Y(double x)`

引数 実数 x
戻り値 実数 $-x - 1 + Ce^x$ の値

```
double Y(double x){ return 2 * exp(x) - x - 1; }
```

③ 関数 `void euler(int N, double x, double y, double X)`

引数 整数 N 刻み区間の数
 実数 x 初期値 $x = x_0$
 実数 y 初期値 $y(x_0)$
 実数 X $y(x)$ の評価対象となる x の値
戻り値 実数 $y(X)$ の計算結果

刻み幅 $h = \frac{1}{N}$ から計算し、 x を X になるまで h ずつ増やしながら $y(x) \approx y + h \times f(x,y)$ を計算し、 $y(x = X)$ の計算結果を返す。

```
double euler(int N, double x, double y, double X){
    int i;
    double h = 1.0 / N;
    for(i = 0; i < N; i++){
        y = y + h * f(x,y);
        x = x + h;
    }
    return y;
}
```

④ 関数 void runge(int N, double x, double y, double X)

引数	整数 N	刻み区間の数
	実数 x	初期値 $x = x_0$
	実数 y	初期値 $y(x_0)$
	実数 X	$y(x)$ の評価対象となる x の値

戻り値 実数 $y(X)$ の計算結果

刻み幅 $h = \frac{1}{N}$ から計算し、 x を X になるまで h ずつ増やしながらルンゲ・クッタ法を用いて $y(x)$ を計算し、 $y(x = X)$ の計算結果を返す。

```
double runge(int N, double x, double y, double X){
    int i;
    double h = 1.0 / N;
    double k1, k2, k3, k4;

    for(i = 0; i < N; i++){
        k1 = h * f(x,y);
        k2 = h * f(x + h / 2, y + k1 / 2);
        k3 = h * f(x + h / 2, y + k2 / 2);
        k4 = h * f(x + h, y + k3);
        y = y + (k1 + 2 * k2 + 2 * k3 + k4) / 6;
        x = x + h;
    }

    return y;
}
```

3. メイン関数

```
int main(int argc, char** argv){
    double x0 = 0, y0 = 1;
    double x = 1, y;
    int i, j, n;
    FILE* output_euler, *output_runge;
```

```

output_euler = fopen("./4.3.euler.dat","w+");
output_runge = fopen("./4.3.runge.dat","w+");
for(i = 0; i <= 4; i++){
    for(j = 2; j <= 10; j++){
        n = j * pow(10,i);
        y = euler(n,x0,y0,x);
        fprintf(output_euler,"%0.18f\t%0.18f\t%0.18f\t%0.18f\n",1.0 / n, y, Y(x), fabs(Y(x) - y));
        y = runge(n,x0,y0,x);
        fprintf(output_runge,"%0.18f\t%0.18f\t%0.18f\t%0.18f\n",1.0 / n, y, Y(x), fabs(Y(x) - y));
    }
}
fclose(output_euler);
fclose(output_runge);
}

```

刻み区間の数 n を 2, 3, ..., 9, 10, 20, ..., 90, 100, 200, ..., 900, 1000, 2000, ..., 10000, 20000, ..., 100000 個として実装し、刻み幅が $\frac{1}{n}$ のときの結果をファイル ./4.3.euler.dat と ./4.3.runge.dat に書き込む。

4. 実行結果

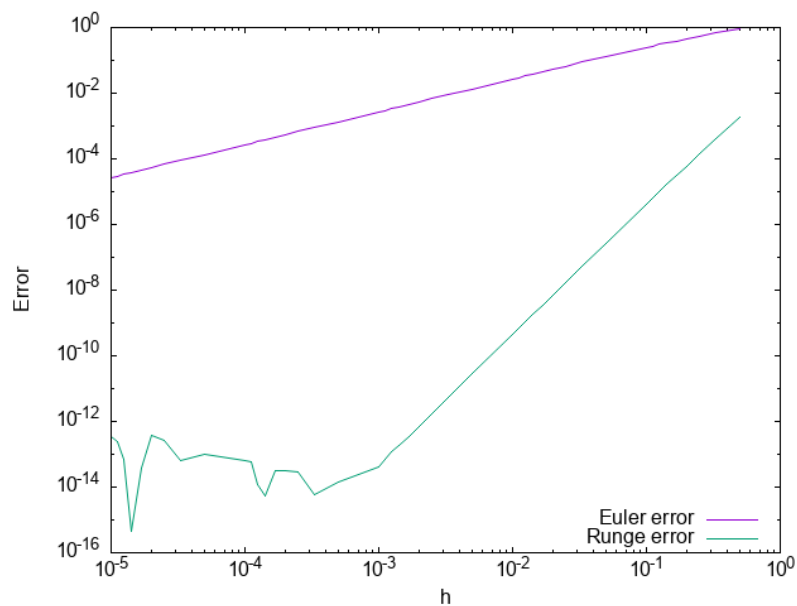


図 3.1 オイラー法とルンゲ・クッタ法の刻み幅による誤差

課題 4.2 と同様にルンゲ・クッタ法による誤差はオイラー法より小さい。刻み幅が大きくなれば両方とも誤差が拡大していくことが分かる。刻み幅が 10^{-3} を超えるとルンゲ・クッタ法による誤差は甚だしく増えていく。逆にオイラー法の増大速度が変わらない。 10^{-5} から 10^{-3} までの間、ルンゲ・クッタ法による誤差の凹凸が見える原因は、打ち切り誤差などの蓄積から誤差生じるからだと考えられる

自由課題

課題 4.2 に加えてホイン法のプログラムを書き，1 階の常微分方程式 $y' = x + y$ をそれぞれの方法で 計算しなさい。また，解析解と比較したときの誤差特性をグラフ化し検討しなさい。

1. ホイン法

ホイン法は微分方程式 $y' = f(x, y)$ の解を求める数値解析法である。

$$\tilde{y}_{i+1} = y_i + hf(x_i, y_i)$$

$$y_{i+1} = y_i + \frac{h}{2} \{f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1})\}$$

y_0 を $x = 0$ における初期値とする。

2. ホイン法関数

引数 実数 x 変数 $x = x_0$ の初期値

実数 y 変数 $y(x_0)$ の初期値

実数 a, b 積分範囲

```
double heun(double x, double y, double a, double b){
    int i;
    double _y;
    double h = (b - a) / N;
    FILE* output = fopen("./heun.dat", "w+");
    for(i = 0; i < N; i++){
        _y = y + h * f(x, y);
        y = y + h * (f(x, y) + f(x + h, _y)) / 2;
        x = x + h;
        fprintf(output, "%.10f\t%.10f\t%.10f\t%.10f\n", x, y, Y(x), fabs(Y(x) - y));
    }
    fclose(output);
    return y;
}
```

3. 実行結果

図 4.1 から、ホイン法による数値解がオイラーより精度がよく、ルンゲ・クッタ法と同様に y の直線に一致していると見える。図 4.2 から見れば、ホイン法による誤差がオイラー法より小さいが、ルンゲ・クッタ法より大きいことが分かる。

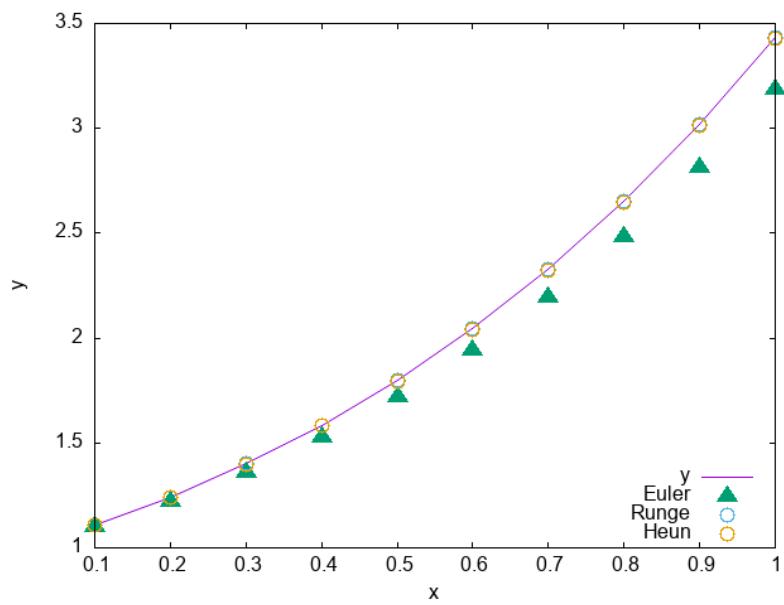


図 4.1 オイラー法とホイン法とルンゲ・クッタ法による計算結果

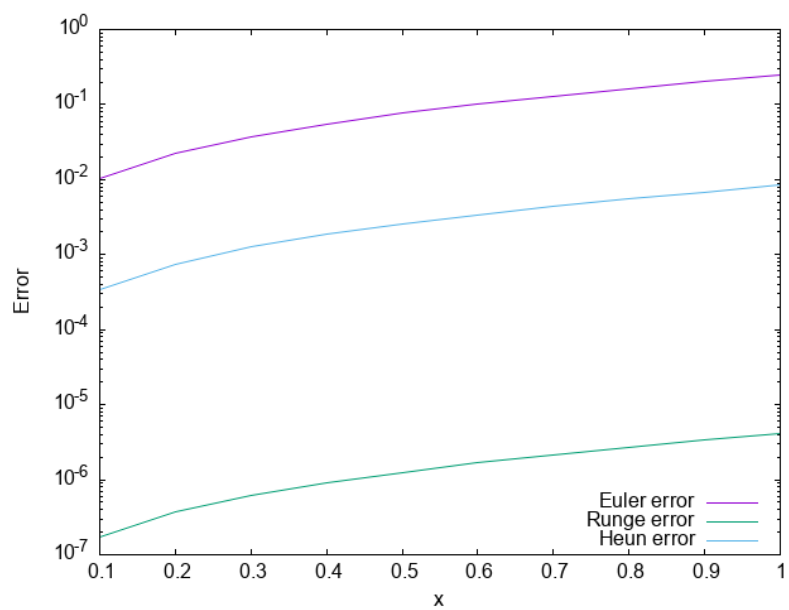


図 4.2 オイラー法とホイン法とルンゲ・クッタ法による計算の誤差